

Place and Route of the MSP430 Hardware Multiplier

Taking the Lab 2 netlist through IC Compiler

Bibek Shrestha | October 2026 | Synopsys IC Compiler, SAED 90 nm

The assignment

Run the IC Compiler flow from the tutorial on a design from the previous lab: floorplan, power plan, placement, clock tree synthesis and routing. Generate and analyze the reports after CTS and after routing.

At a glance

	Lab 2 (DC)	After CTS	After routing
Library corner	typ	max	max
Wires	none	estimated	routed + extracted
Clock	7 ns	7 ns	7 ns
Critical path	6.91 ns	[ns]	[ns]
Setup slack	0.00 ns	[ns]	[ns]
Violating paths	0	[]	[]
Hold violations		[]	[]
Cells	881	[]	[]
Cell area	12677.53	[]	[]
Utilization		[%]	[%]
Open nets / DRCs			[] / []

Table 1: Lab 2 synthesis next to the two ICC checkpoints.

1 The design

The same `mult430` from Lab 2: a memory-mapped 16×16 multiplier (MPY, MPYS, MAC, MACS) on a shared tri-state data bus. It has two submodules, the control FSM `u_ctrl` and the datapath `u_dp`. The 32-bit adder inside the datapath was the critical path in Lab 2.

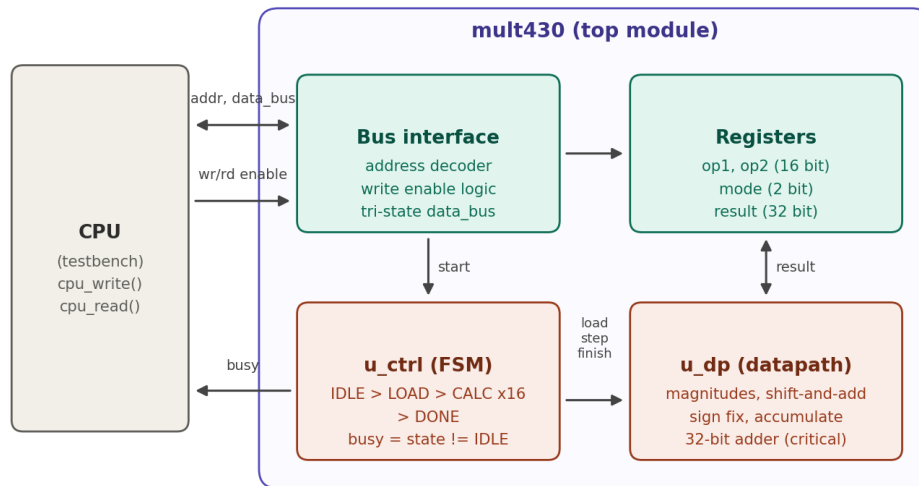


Figure 1: Architecture from Lab 2.

Ports: `clk`, `reset_n`, `addr[15:0]`, `data_bus[15:0]` (inout), `write_enable`, `read_enable`, `busy`. That is 52 signal pins, compared with 7 for the gray counter.

2 Script changes

The script is the tutorial flow with new names, plus one addition.

Tutorial	This design
<code>gray4bcntLIB</code>	<code>MULT_LIB</code>
<code>gray_counter.ddc</code> , top <code>gray_counter</code>	<code>assignment3.ddc</code> , top <code>mult430</code>
<code>gray_counter.sdc</code> (5 ns)	<code>assignment3.sdc</code> (7 ns)
checkpoints <code>gray4bcnt_*</code>	<code>MULT_init</code> , <code>MULT_cts_opt</code> , <code>MULT_route</code> <code>set_dont_use</code> on <code>BSLEX1/2/4</code>

Table 2: Differences from the tutorial script.

In Lab 2 the BSLEX tri-state cells gave wrong gate-level results, so Design Compiler was told not to use them and picked `TNBUFFX` instead. `place_opt` and `route_opt` can resize or swap cells while fixing timing, and they do not know about the Lab 2 ban, so the same `set_dont_use` lines are repeated in ICC.

3 Flow, stage by stage

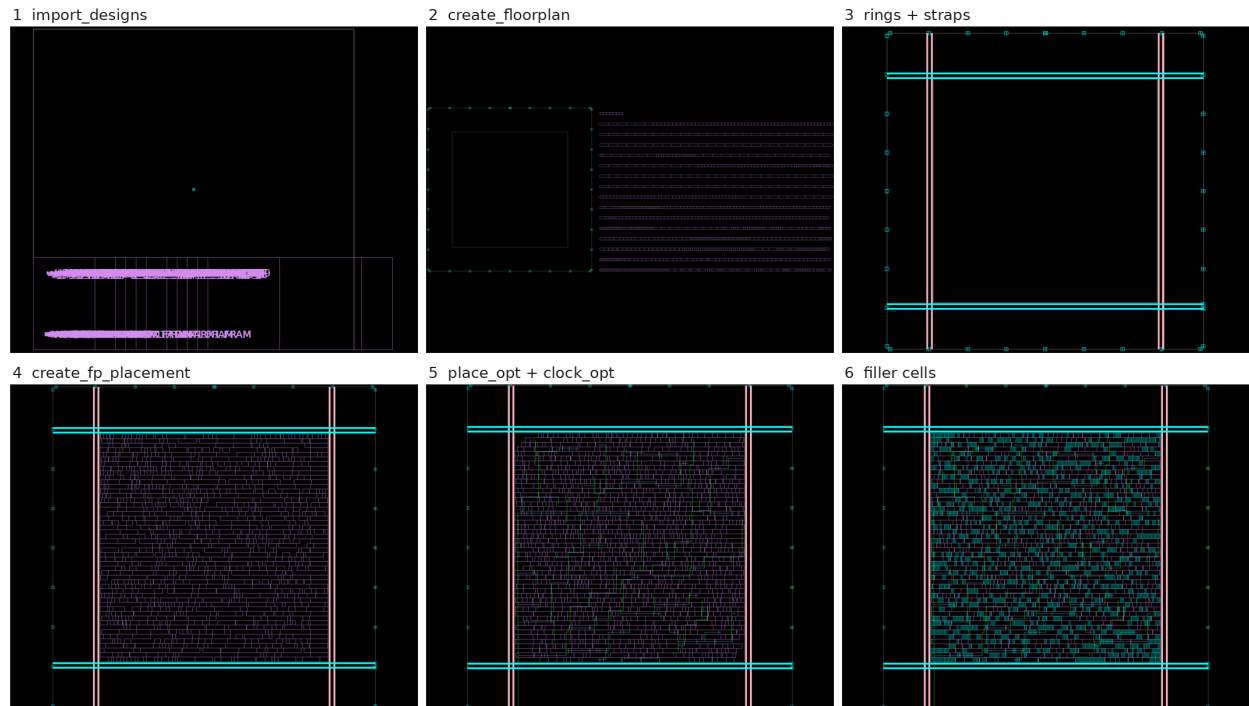


Figure 2: (1) After import the cells are stacked outside the core. (2) The floorplan sizes the core; the cells wait in rows beside it. (3) VSS/VDD rings and straps. (4) Rough placement fills the core. (5) After `clock_opt` the first routed nets (green) are the clock tree. (6) Filler cells close the row gaps.

Compared with the gray counter, the core is much larger: the same 0.6 utilization now has to hold 881 cells instead of 15, so the die grows while the $30\mu\text{m}$ ring margin stays the same.

4 Routed layout

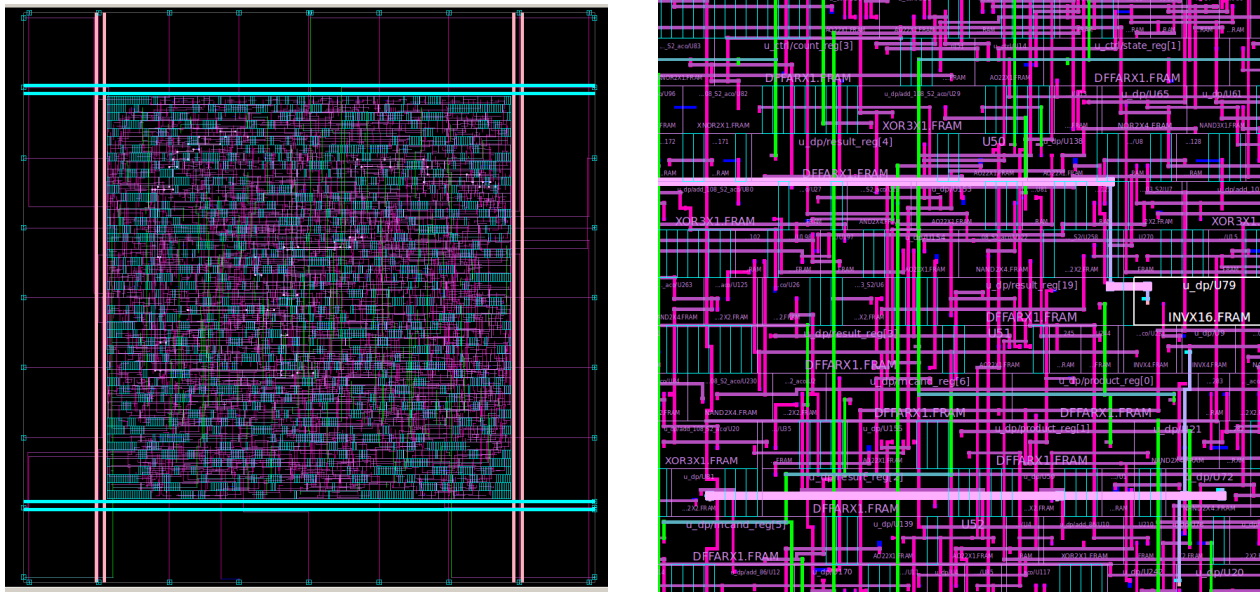


Figure 3: Left: the full routed block. Right: a zoom. The instance names keep the RTL hierarchy: u_ctrl/state_reg, u_ctrl/count_reg, u_dp/product_reg, u_dp/result_reg, u_dp/mcand_reg, and u_dp/add_108_52 for the adder Design Compiler built.

5 Reports

Report	What I checked	Result
MULT_route_qor.rpt	slack, violating paths	☐
MULT_route.setup.rpt	worst setup path	[start → end, slack]
MULT_route.hold.rpt	worst hold path	☐
MULT_route_util.rpt	standard cell utilization	☐
verify_zrt_route	open nets, DRCs	☐

Table 3: Post-route reports.

Why the timing is different from Lab 2

Lab 2 had exactly 0.00 ns of slack at 7 ns, but that was with the *typical* library and no wires. ICC checks the *max* (slow) corner and, after routing, adds real wire R and C. Both make the 32-bit carry chain slower, so the post-route slack is [ns].

6 Inside the cells

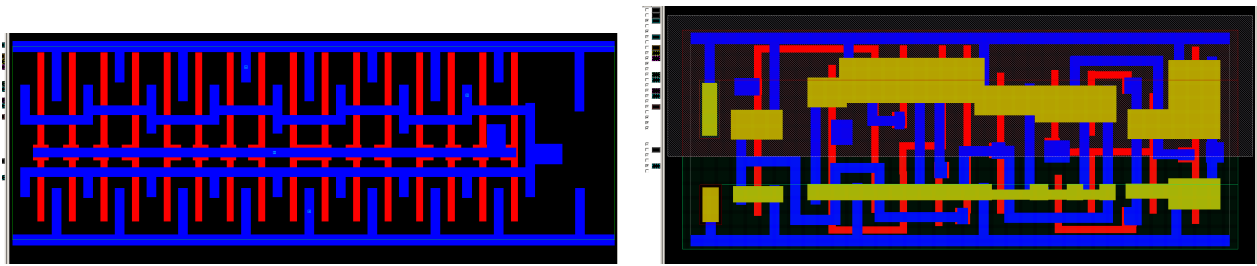


Figure 4: Left: INVM16 (instance u_dp/U79), FRAM view. It is 16 inverters in parallel: the repeated red poly fingers share one input and one output, which gives the high drive strength. Right: X0R3X1, CEL view, with the N-well (hatched) on top for the PMOS and the NMOS below.

7 What I learned

- Physical design is where the real timing shows up. A path with 0.00 ns slack after synthesis has no room left for the slow corner or for wire delay.
- The hierarchy (u_ctrl, u_dp) survives into the layout, which makes it easy to find a register in the GUI.
- Constraints set in Design Compiler, like set_dont_use, are not carried into ICC automatically.
- A larger design takes much longer to draw and route; “REDRAW INTERRUPTED” in the GUI is just the display giving way to the running command.

A Source files

RTL: src/assignment3.sv

```

/*****
MSP430 Hardware Multiplier Accelerator
Modes: MPY (0x0130), MPYS (0x0132), MAC (0x0134), MACS (0x0136)
OP2 write (0x0138) starts the operation
Result: RESL (0x013A) = result[15:0], RESH (0x013C) = result[31:16]
Multiply method: 16-cycle shift-and-add on magnitudes, sign fixed at end
*****/

//=====
// Control unit: FSM IDLE -> LOAD -> CALC (16 cycles) -> DONE
//=====

`timescale 1ns/1ps

module mult430_ctrl (
    input logic clk,
    input logic reset_n,
    input logic start,      // pulse: OP2 was written
    output logic load,      // load magnitudes into datapath
    output logic step,      // do one shift-and-add step
    output logic finish,    // apply sign, update result
    output logic busy
);
    typedef enum logic [1:0] {IDLE = 2'b00, LOAD = 2'b01,

```

```

        CALC = 2'b10, DONE = 2'b11} state_t;
state_t state, next_state;
logic [3:0] count;

// state register
always_ff @(posedge clk or negedge reset_n)
    if (!reset_n) state <= IDLE;
    else          state <= next_state;

// step counter (counts 0..15 while in CALC)
always_ff @(posedge clk or negedge reset_n)
    if (!reset_n) count <= 4'd0;
    else if (state == CALC) count <= count + 4'd1;
    else count <= 4'd0;

// next-state logic
always_comb begin
    next_state = state;
    case (state)
        IDLE: if (start) next_state = LOAD;
        LOAD: next_state = CALC;
        CALC: if (count == 4'd15) next_state = DONE;
        DONE: next_state = IDLE;
    endcase
end

// outputs
assign load   = (state == LOAD);
assign step   = (state == CALC);
assign finish = (state == DONE);
assign busy   = (state != IDLE);
endmodule

//=====
// Datapath: magnitudes, shift-and-add, sign fix, accumulate
//=====
module mult430_dp (
    input logic clk,
    input logic reset_n,
    input logic [15:0] op1,
    input logic [15:0] op2,
    input logic [1:0] mode, // mode[0] = signed, mode[1] = accumulate
    input logic load,
    input logic step,
    input logic finish,
    output logic [31:0] result
);
    logic [31:0] mcand; // multiplicand, shifts left each step
    logic [15:0] mplier; // multiplier, shifts right each step
    logic [31:0] product; // running unsigned product
    logic neg; // final product must be negated
    logic [15:0] mag1, mag2; // operand magnitudes
    logic [31:0] signed_prod; // product with sign applied

    // magnitude = two's complement negate if signed mode and negative
    assign mag1 = (mode[0] && op1[15]) ? (~op1 + 16'd1) : op1;
    assign mag2 = (mode[0] && op2[15]) ? (~op2 + 16'd1) : op2;

    // apply sign at the end
    assign signed_prod = neg ? (~product + 32'd1) : product;

    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) begin
            mcand <= 32'd0;
            mplier <= 16'd0;

```

```

        product <= 32'd0;
        neg      <= 1'b0;
        result   <= 32'd0;
    end
    else if (load) begin
        mcand    <= {16'd0, mag1};
        mplier   <= mag2;
        product  <= 32'd0;
        neg      <= mode[0] & (op1[15] ^ op2[15]);
    end
    else if (step) begin
        if (mplier[0]) product <= product + mcand;
        mcand    <= mcand << 1;
        mplier   <= mplier >> 1;
    end
    else if (finish) begin
        if (mode[1]) result <= result + signed_prod; // MAC / MACS
        else        result <= signed_prod;           // MPY / MPYS
    end
endmodule

//=====
// Top module: bus interface, registers, tri-state data bus
//=====
module mult430 (
    input logic      clk,
    input logic      reset_n,
    input logic [15:0] addr,
    inout wire [15:0] data_bus,
    input logic      write_enable,
    input logic      read_enable,
    output logic      busy
);
    localparam logic [15:0] ADDR_MPY  = 16'h0130;
    localparam logic [15:0] ADDR_MPYS = 16'h0132;
    localparam logic [15:0] ADDR_MAC  = 16'h0134;
    localparam logic [15:0] ADDR_MACS = 16'h0136;
    localparam logic [15:0] ADDR_OP2  = 16'h0138;
    localparam logic [15:0] ADDR_RESL  = 16'h013A;
    localparam logic [15:0] ADDR_RESR  = 16'h013C;

    logic [15:0] op1, op2;
    logic [1:0] mode;
    logic [31:0] result;
    logic        wr_ok, start, load, step, finish;
    logic        rd_en;
    logic [15:0] rd_data;

    // a write is accepted only when not busy and not reading
    assign wr_ok = write_enable && !read_enable && !busy;
    assign start = wr_ok && (addr == ADDR_OP2);

    // write registers
    always_ff @(posedge clk or negedge reset_n)
        if (!reset_n) begin
            op1 <= 16'd0;
            op2 <= 16'd0;
            mode <= 2'b00;
        end
        else if (wr_ok) begin
            case (addr)
                ADDR_MPY: begin op1 <= data_bus; mode <= 2'b00; end
                ADDR_MPYS: begin op1 <= data_bus; mode <= 2'b01; end
                ADDR_MAC:  begin op1 <= data_bus; mode <= 2'b10; end
                ADDR_MACS: begin op1 <= data_bus; mode <= 2'b11; end
            endcase
        end
endmodule

```

```

        ADDR_OP2: op2 <= data_bus;
        default: ;
    endcase
end

// read mux
always_comb begin
    rd_en = 1'b0;
    rd_data = 16'h0000;
    if (read_enable && !write_enable) begin
        if (addr == ADDR_RESL) begin
            rd_en = 1'b1;
            rd_data = result[15:0];
        end
        else if (addr == ADDR_RESR) begin
            rd_en = 1'b1;
            rd_data = result[31:16];
        end
    end
end

// tri-state driver: drive only on a valid read, else release the bus
assign data_bus = rd_en ? rd_data : 16'bz;

// control unit
mult430_ctrl u_ctrl (
    .clk      (clk),
    .reset_n  (reset_n),
    .start    (start),
    .load     (load),
    .step     (step),
    .finish   (finish),
    .busy     (busy)
);

// datapath
mult430_dp u_dp (
    .clk      (clk),
    .reset_n  (reset_n),
    .op1      (op1),
    .op2      (op2),
    .mode     (mode),
    .load     (load),
    .step     (step),
    .finish   (finish),
    .result   (result)
);
endmodule

```

Testbench: src/assignment3_tb.sv

```

/*****
Testbench for MSP430 Hardware Multiplier (mult430)
The testbench acts as the CPU:
    cpu_write(addr, data) -> drives data_bus, pulses write_enable
    cpu_read (addr, data) -> releases data_bus, pulses read_enable
A software model computes the expected result for every test.
*****/

`timescale 1ns/1ps

```

```

module mult430_tb;

    // address map
    localparam logic [15:0] ADDR_MPY  = 16'h0130;
    localparam logic [15:0] ADDR_MPYS = 16'h0132;
    localparam logic [15:0] ADDR_MAC  = 16'h0134;
    localparam logic [15:0] ADDR_MACS = 16'h0136;
    localparam logic [15:0] ADDR_OP2  = 16'h0138;
    localparam logic [15:0] ADDR_RESL  = 16'h013A;
    localparam logic [15:0] ADDR_RESR  = 16'h013C;

    // DUT signals
    logic      clk;
    logic      reset_n;
    logic [15:0] addr;
    wire  [15:0] data_bus;
    logic      write_enable;
    logic      read_enable;
    logic      busy;

    // CPU side tri-state driver
    logic      tb_drive;
    logic [15:0] tb_data;
    assign data_bus = tb_drive ? tb_data : 16'bz;

    // bookkeeping
    logic [31:0] model;          // expected contents of the result register
    integer      errors;
    integer      tests;

    // DUT
    mult430 dut (
        .clk      (clk),
        .reset_n   (reset_n),
        .addr      (addr),
        .data_bus  (data_bus),
        .write_enable (write_enable),
        .read_enable (read_enable),
        .busy      (busy)
    );

    // clock: 10 ns period
    always #5 clk = ~clk;

    //-----
    // CPU write: set signals on negedge, DUT captures on posedge
    //-----
    task automatic cpu_write(input logic [15:0] a, input logic [15:0] d);
        @(negedge clk);
        addr      = a;
        tb_data    = d;
        tb_drive   = 1'b1;
        write_enable = 1'b1;
        @(negedge clk);
        write_enable = 1'b0;
        tb_drive    = 1'b0;
    endtask

    //-----
    // CPU read: release bus, sample half a cycle later
    //-----
    task automatic cpu_read(input logic [15:0] a, output logic [15:0] d);
        @(negedge clk);
        addr      = a;

```

```

    tb_drive    = 1'b0;
    read_enable = 1'b1;
    @(posedge clk);
    d = data_bus;
    @(negedge clk);
    read_enable = 1'b0;
endtask

//-----
// wait until the multiplier is no longer busy
//-----
task automatic wait_done();
    @(negedge clk);
    while (busy) @(negedge clk);
endtask

//-----
// expected product for a given mode
//-----
function automatic logic [31:0] expected_product(input logic [1:0] m,
                                                    input logic [15:0] a,
                                                    input logic [15:0] b);

    logic signed [31:0] sa, sb;
    if (m[0]) begin
        sa = $signed(a);           // sign-extend to 32 bits
        sb = $signed(b);
        expected_product = sa * sb;
    end
    else
        expected_product = {16'd0, a} * {16'd0, b};
endfunction

//-----
// run one operation and check RESL / RESH
//-----
task automatic run_op(input string name, input logic [15:0] mode_addr,
                     input logic [15:0] a, input logic [15:0] b);

    logic [1:0] m;
    logic [15:0] lo, hi;
    logic [31:0] got;

    m = mode_addr[2:1];           // 0130->00, 0132->01, 0134->10, 0136->11
    if (m[1]) model = model + expected_product(m, a, b);
    else
        model = expected_product(m, a, b);

    cpu_write(mode_addr, a);
    cpu_write(ADDR_OP2, b);
    wait_done();
    cpu_read(ADDR_RESL, lo);
    cpu_read(ADDR_RESB, hi);
    got = {hi, lo};

    tests = tests + 1;
    if (got == model)
        $display("PASS %-5s a=0x%h b=0x%h result=0x%h", name, a, b, got);
    else begin
        $display("FAIL %-5s a=0x%h b=0x%h result=0x%h expected=0x%h",
                 name, a, b, got, model);
        errors = errors + 1;
    end
endtask

//-----
// test sequence
//-----

```

```

initial begin
    clk          = 1'b0;
    reset_n      = 1'b0;
    addr         = 16'h0000;
    write_enable = 1'b0;
    read_enable  = 1'b0;
    tb_drive     = 1'b0;
    tb_data      = 16'h0000;
    model        = 32'd0;
    errors       = 0;
    tests        = 0;

    repeat (2) @(posedge clk);
    @(negedge clk);
    reset_n = 1'b1;

    $display("\n--- Unsigned multiply (MPY) ---");
    run_op("MPY", ADDR_MPY, 16'd3, 16'd5);           // 15
    run_op("MPY", ADDR_MPY, 16'hFFFF, 16'hFFFF);    // 0xFFFE0001
    run_op("MPY", ADDR_MPY, 16'd1234, 16'd0);        // 0

    $display("\n--- Signed multiply (MPYS) ---");
    run_op("MPYS", ADDR_MPYS, 16'hFFFD, 16'd5);      // -3 * 5 = -15
    run_op("MPYS", ADDR_MPYS, 16'hFFFC, 16'hFFFA);  // -4 * -6 = 24
    run_op("MPYS", ADDR_MPYS, 16'h8000, 16'h8000);   // -32768^2 = 0x40000000
    run_op("MPYS", ADDR_MPYS, 16'd0, 16'hFFFF);      // clear result to 0

    $display("\n--- Unsigned multiply-accumulate (MAC) ---");
    run_op("MAC", ADDR_MAC, 16'd2, 16'd10);          // 0 + 20 = 20
    run_op("MAC", ADDR_MAC, 16'd4, 16'd10);          // 20 + 40 = 60

    $display("\n--- Signed multiply-accumulate (MACS) ---");
    run_op("MACS", ADDR_MACS, 16'hFFFE, 16'd10);     // 60 - 20 = 40
    run_op("MACS", ADDR_MACS, 16'hFFFB, 16'hFFFB);   // 40 + 25 = 65

    $display("\n--- Write while busy must be ignored ---");
    model = expected_product(2'b00, 16'd7, 16'd7);   // expect 49
    cpu_write(ADDR_MPY, 16'd7);
    cpu_write(ADDR_OP2, 16'd7);
    cpu_write(ADDR_MAC, 16'd100);                    // MAC write while busy -> must be
    ignored
    wait_done();
    begin
        logic [15:0] lo, hi;
        cpu_read(ADDR_RESL, lo);
        cpu_read(ADDR_RESH, hi);
        tests = tests + 1;
        if ({hi, lo} == model)
            $display("PASS BUSY result=0x%h", {hi, lo});
        else begin
            $display("FAIL BUSY result=0x%h expected=0x%h", {hi, lo}, model);
            errors = errors + 1;
        end
    end
end

$display("\n--- Bus released when not reading ---");
begin
    logic [15:0] v;
    // 1) idle: nobody should drive the bus
    @(negedge clk);
    tests = tests + 1;
    if (data_bus == 16'hzzzz)
        $display("PASS BUSZ idle bus=0x%h", data_bus);
    else begin
        $display("FAIL BUSZ idle bus=0x%h expected=zzzz", data_bus);
    end
end

```

```

        errors = errors + 1;
    end
    // 2) read a non-result address: multiplier must not drive
    cpu_read(ADDR_OP2, v);
    tests = tests + 1;
    if (v == 16'hzzzz)
        $display("PASS BUSZ read 0x0138 bus=0x%h", v);
    else begin
        $display("FAIL BUSZ read 0x0138 bus=0x%h expected=zzzz", v);
        errors = errors + 1;
    end
end

$display("\n---- Mode switching ----");
run_op("MAC", ADDR_MAC, 16'd3, 16'd3); // 49 + 9 = 0x0000003A
run_op("MPYS", ADDR_MPYS, 16'hFFFF, 16'hFFFF); // -1 * -1 = 0x00000001
run_op("MPY", ADDR_MPY, 16'hFFFF, 16'd2); // 65535 * 2 = 0x0001FFFE
run_op("MACS", ADDR_MACS, 16'hFFFF, 16'd3); // 0x1FFFE - 3 = 0x0001FFFB
run_op("MAC", ADDR_MAC, 16'hFFFF, 16'd1); // + 65535 = 0x0002FFFA

$display("\n---- Accumulate overflow / wrap-around ----");
run_op("MPY", ADDR_MPY, 16'hFFFF, 16'hFFFF); // 0xFFFE0001
run_op("MAC", ADDR_MAC, 16'hFFFF, 16'hFFFF); // 0x1_FFFC0002 -> 0xFFFC0002
run_op("MPYS", ADDR_MPYS, 16'd0, 16'd0); // clear to 0
run_op("MACS", ADDR_MACS, 16'hFFFF, 16'd1); // 0 - 1 -> 0xFFFFFFFF
run_op("MAC", ADDR_MAC, 16'd1, 16'd1); // 0xFFFFFFFF + 1 -> 0x00000000

$display("\n---- Reset in the middle of an operation ----");
run_op("MPY", ADDR_MPY, 16'h1234, 16'h0010); // non-zero first: 0x00012340
begin
    logic [15:0] lo, hi;
    logic busy_before;
    cpu_write(ADDR_MPY, 16'd7);
    cpu_write(ADDR_OP2, 16'd9); // start 7 * 9
    repeat (5) @(negedge clk); // let it run a few steps
    busy_before = busy;
    reset_n = 1'b0; // async reset, no clock edge
    #1;
    tests = tests + 1;
    if (busy_before == 1'b1 && busy == 1'b0)
        $display("PASS RST busy was 1, dropped to 0 immediately on reset");
    else begin
        $display("FAIL RST busy before=%b after=%b (expected 1 then 0)",
            busy_before, busy);
        errors = errors + 1;
    end
    repeat (2) @(negedge clk);
    reset_n = 1'b1; // release reset
    model = 32'd0; // reset clears the result
    cpu_read(ADDR_RESL, lo);
    cpu_read(ADDR_RESR, hi);
    tests = tests + 1;
    if ({hi, lo} == model)
        $display("PASS RST result cleared to 0x%h", {hi, lo});
    else begin
        $display("FAIL RST result=0x%h expected=0x%h", {hi, lo}, model);
        errors = errors + 1;
    end
end
end

run_op("MPY", ADDR_MPY, 16'd6, 16'd7); // works after reset: 0x0000002A

$display("\n---- Random tests (200 operations, all modes, seed 430) ----");
begin

```

```

logic [15:0] ra, rb, lo, hi, maddr;
logic [1:0] m;
integer i, rand_err;
integer cnt_mpy, cnt_mpys, cnt_mac, cnt_macs;
void'($urandom(430)); // fixed seed -> repeatable
rand_err = 0;
cnt_mpy = 0; cnt_mpys = 0; cnt_mac = 0; cnt_macs = 0;
for (i = 0; i < 200; i = i + 1) begin
    // random mode: 0130, 0132, 0134, 0136
    m = $urandom_range(0, 3);
    maddr = ADDR_MPY + {13'd0, m, 1'b0};
    case (m)
        2'b00: cnt_mpy = cnt_mpy + 1;
        2'b01: cnt_mpys = cnt_mpys + 1;
        2'b10: cnt_mac = cnt_mac + 1;
        2'b11: cnt_macs = cnt_macs + 1;
    endcase
    // operands, biased toward edge values
    case ($urandom_range(0, 7))
        0: ra = 16'h0000; 1: ra = 16'hFFFF;
        2: ra = 16'h8000; 3: ra = 16'h7FFF;
        default: ra = $urandom;
    endcase
    case ($urandom_range(0, 7))
        0: rb = 16'h0000; 1: rb = 16'hFFFF;
        2: rb = 16'h8000; 3: rb = 16'h7FFF;
        default: rb = $urandom;
    endcase
    // expected value
    if (m[1]) model = model + expected_product(m, ra, rb);
    else model = expected_product(m, ra, rb);
    // run on the DUT
    cpu_write(maddr, ra);
    cpu_write(ADDR_OP2, rb);
    wait_done();
    cpu_read(ADDR_RES_L, lo);
    cpu_read(ADDR_RES_H, hi);
    tests = tests + 1;
    if ({hi, lo} != model) begin
        rand_err = rand_err + 1;
        errors = errors + 1;
        if (rand_err <= 10)
            $display("FAIL RAND #0d mode=%b a=0x%h b=0x%h result=0x%h expected=0x%h"
,
                        i, m, ra, rb, {hi, lo}, model);
    end
end
$display("RAND 200 ops (MPY %0d, MPYS %0d, MAC %0d, MACS %0d): %0d failures",
        cnt_mpy, cnt_mpys, cnt_mac, cnt_macs, rand_err);
end

$display("\n=== %0d tests, %0d errors ===", tests, errors);
if (errors == 0) $display("All tests completed successfully\n");
$finish;
end

// waveform dump
initial begin
    $dumpfile("assignment3.dump");
    $dumpvars(0, mult430_tb);
end

```

```
endmodule
```

ICC script: src/icc_mult.tcl

```
# icc_mult.tcl : Lab 3, Assignment 3 (MSP430 hardware multiplier, top = mult430)
# Run from icc_assignment3/, one level below output/ and constraints/:
#   icc_shell -shared_license -f icc_mult.tcl | tee icc_mult.log

gui_start

set_app_var search_path /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/
synopsys/models/
set_app_var target_library "saed90nm_max.db"
set_app_var link_library "* saed90nm_max.db"

create_mw_lib -technology "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/milkyway/
saed90nm_icc_1p9m.tf" -mw_reference_library "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Digital_Standard_cell_Library/process/astro/fram/saed90nm/" "MULT_LIB"

open_mw_lib MULT_LIB

import_designs "../output/assignment3.ddc" -format ddc -top "mult430" -cel "mult430"

read_sdc ../constraints/assignment3.sdc

# same as Lab 2: keep ICC from using the BSLEX tri-state cells when it resizes or adds gates
set_dont_use [get_lib_cells */BSLEX1]
set_dont_use [get_lib_cells */BSLEX2]
set_dont_use [get_lib_cells */BSLEX4]

set_tlu_plus_files -max_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/starrcxt/
tluplus/saed90nm_1p9m_1t_Cmax.tluplus" -min_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Technology_Kit/starrcxt/tluplus/saed90nm_1p9m_1t_Cmin.tluplus" -tech2itf_map "/apps/designlib/
SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/process/astro/tech/tech2itf.map"

save_mw_cel -as MULT_init

create_floorplan -core_utilization 0.6 -start_first_row -left_io2core "30" -bottom_io2core "30" -
right_io2core "30" -top_io2core "30"

derive_pg_connection -power_net "VDD" -power_pin "VDD" -ground_net "VSS" -ground_pin "VSS" -
create_ports "top"

create_rectangular_ring -nets {VSS} -left_offset 0.5 -left_segment_layer M6 -left_segment_width
1.0 -extend_ll -extend_lh -right_offset 0.5 -right_segment_layer M6 -right_segment_width 1.0 -
extend_rl -extend_rh -bottom_offset 0.5 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
extend_bl -extend_bh -top_offset 0.5 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
extend_th

create_rectangular_ring -nets {VDD} -left_offset 1.8 -left_segment_layer M6 -left_segment_width
1.0 -extend_ll -extend_lh -right_offset 1.8 -right_segment_layer M6 -right_segment_width 1.0 -
extend_rl -extend_rh -bottom_offset 1.8 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
extend_bl -extend_bh -top_offset 1.8 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
extend_th

create_power_strap -nets {VSS} -layer M6 -direction vertical -width 3
create_power_strap -nets {VDD} -layer M6 -direction vertical -width 3

create_fp_placement

place_opt
clock_opt
```

```
save_mw_cel -as MULT_cts_opt

report_placement_utilization > ../output/MULT_cts_util.rpt
report_qor > ../output/MULT_cts_qor.rpt
report_timing -delay max -max_paths 5 > ../output/MULT_cts.setup.rpt
report_timing -delay min -max_paths 5 > ../output/MULT_cts.hold.rpt

insert_stdcell_filler -cell_with_metal "SHFILL2" -cell_without_metal "SHFILL2" -connect_to_power {
    VDD} -connect_to_ground {VSS}

route_opt
verify_zrt_route

report_placement_utilization > ../output/MULT_route_util.rpt
report_qor > ../output/MULT_route_qor.rpt
report_timing -delay max -max_paths 5 > ../output/MULT_route.setup.rpt
report_timing -delay min -max_paths 5 > ../output/MULT_route.hold.rpt

extract_rc -coupling_cap
write_parasitics -format SBPF -output ../output/MULT.output.sbpf
write_verilog ../output/MULT_LIB.output.v

save_mw_cel -as MULT_route
```